OpenWorld

VERIFIED CODE WORLD MODELS

build · optimize · deploy

Hybrid Perceptual World Models: World-Time Compute for Interactive Reasoning (ARC-AGI-3)

James Schwoebel[1,*] • Ingrida Semenec, PhD[1] • Jenia Rousseva, PhD[1] • Marcos Ortiz, PhD[1] • Collin Overbay[1] • Manish Bhatt[4,6] • Rome Thorstenson[3] • Martin G. Frasch, MD, PhD[2,5]

[1] Quome, Inc. [2] University of Washington [3] Rafter Labs, Inc. [4] OWASP [5] Health Stream Analytics, LLC
[6] IEEE Computer Society [*] Corresponding author

ABSTRACT

ARC-AGI-3 is the ARC Prize Foundation's interactive-reasoning benchmark: an agent dropped into an unfamiliar grid game must discover the rules *by acting* and reach goals over many steps; frontier agents score near zero. We study what a verified-world-model approach can and cannot do here, and our central contribution is a **diagnosis**: the bottleneck is not modeling but *goal discovery*, and specifically ARC-AGI-3 wins are **goal-as-procedure**, not goal-as-state.

(1) Modeling is largely solved. All 25 public games are replay-deterministic (1.00), so their dynamics are exactly code-expressible; a frontier synthesizer (Claude) writes verified `predict(frame,action)` code reaching 49% mean held-out exact-match on the real-dynamics games (vs. 12% for a local model), with 3 near-perfect (≥90%) models. *(2) The wall.* Despite these models, **four principled goal-discovery methods**—atomic objectives, frontier-LLM hypotheses, Bayesian sub-world planning, and typed navigation—solve *zero* of the hard games, several *even on a fidelity-1.0 model*: a goal scored over a single *state* cannot express an ordered *procedure*, which is why undirected search occasionally stumbles into a win while goal-conditioned planning never does. *(3) Cracking the wall with executable world models.* The diagnosis says *undirected*, observation-only goal discovery fails. The remedy is the executable-world-model recipe: per game, recover the dynamics and *reason the win condition*, build a **verified source-faithful simulator**, *search* for the winning procedure, and *replay-verify* it against the real engine. Reproduced in OpenWorld—each solver a serveable verified `World`—this completes **23/25 games in full** (177/183 levels; Fig. 1, Table 2; atlas cards in Fig. 2), **surpassing the prior executable-world-model state of the art of 15/25** [4] on full-game completion. *And the solutions are efficient:* scored by the official per-level action metric against the human baselines the environment ships, they average **95.8** (22/25 at the human-efficiency cap) versus baseline1's 58—human-competitive on the leaderboard's own efficiency axis (Fig. 3).

We report this honestly. Solves are *offline* with *unbounded resets* and deterministic replay-verification; we do not report RHAE (the leaderboard's live action-efficiency metric), so the comparison is on full-game *completion*, not efficiency. Three games remain partial (`bp35` (7/9), `lf52` (6/10)). For the weaker ≥1-level union, random play alone clears 11/25 under this protocol, so we also report the more

honest 14/25 **above random**. **Contributions:** (i) surpassing the executable-world-model SOTA on full-game completion (23/25, 177/183 levels); (ii) the goal-as-procedure diagnosis explaining why *undirected* discovery fails (four methods, 0 on perfect models) and why explicit per-level search is needed; (iii) the strategy landscape as a research guide; and (iv) an OpenWorld reproduction of the recipe in which every solver is a serveable, verified `World`.

Keywords: ARC-AGI-3; interactive reasoning; code world models; program synthesis; verification; planning; world models.

1 Introduction

The interactive frontier. ARC-AGI-3 [2] is the latest in the series. ARC-AGI-1 and -2 measure abstraction from static input–output grids; ARC-AGI-3 moves to *interactive* environments where the rules and goals are revealed only through action. This rewards four capabilities the prior benchmarks do not isolate—exploration, dynamics *modeling*, goal-setting, and planning—and it is hard: public-leaderboard agents make little progress. Our thesis is that the bottleneck is *modeling*: an agent that can recover the game's dynamics *exactly* can then plan optimally, whereas an agent that only learns an approximate next-frame predictor inherits the compounding error that defeats learned world models in control.

This paper. We port OpenWorld's verified-code-world-model recipe to ARC-AGI-3 and use it to *diagnose* the benchmark. Our contributions, in order of importance: **(i) the goal-as-procedure diagnosis**—four principled goal-discovery methods (atomic objectives, frontier-LLM hypotheses, Bayesian sub-worlds, typed navigation) solve *zero* hard games, several even on a fidelity-1.0 model, because a goal scored over a single *state* cannot express an ordered *procedure*; **(ii) the strategy landscape** (Fig. 7)—a guide to what does and does not solve interactive reasoning, with the negatives as first-class results; **(iii) an honest above-random measure**—under the offline unbounded-reset protocol random play reaches $\geq$1 level on 11/25, so we report 14/25 **above random** rather than the inflated $\geq$1-level union; and **(iv) a new full-game state of the art**—the executable-world-model recipe (recover dynamics, reason the win condition, build a verified per-game simulator, search, and replay-verify) fully completes **23/25 games** (177/183 levels), each a serveable verified OpenWorld `World` (§2), **surpassing the prior 15/25** [4] on full-game completion. Solves are offline with unbounded resets and replay-verified; we compare on completion, not RHAE efficiency.

Two numbers, read carefully (25/25 $\geq$1-level vs. 23/25 full). ARC-AGI-3 games have *multiple* levels (6–10 each), so "solved" is ambiguous. We report *both* ways of counting:

- **$\geq$1-level reachability = 25/25.** We complete *at least one* level of every public game. *What it shows:* a win is *reachable* at all. *Why it is weak:* level 1 is often a tutorial; our solves are *offline* with *unbounded resets*, so random play alone already clears 11/25 (we beat random on 14/25). We therefore do not lead with this number.
- **Full-game completion = 23/25.** We complete *every* level of 23 of the 25 games (177/183 levels total); the remaining three are partial (`bp35` (7/9), `lf52` (6/10)). *What it shows:* the metric the leaderboard and prior work use—**surpassing the prior SOTA of** 15/25 [4]. *How:* the executable-world-model recipe (§2), with per-level search supplying the multi-step *goal composition* that undirected discovery cannot.

Table 1: Prior art on ARC-AGI-3 (public set). RHAE = relative human action efficiency; metrics differ across systems, so figures are not directly comparable (full *games* vs. *levels* vs. ≥1-level). Frontier-agent and SOTA figures as reported in the cited works and the leaderboard. †Our action-efficiency is the official per-level score of our *executed* solutions against the shipped human baselines (offline; §2.1), not a live-protocol RHAE entry.

System	Approach	Reported result
Frontier LLM agents	direct agent (GPT-5.4, Gemini 3.1, Opus 4.6)	< 0.4% [2]
baseline1 [4]	executable world models (GPT-5.5)	**15/25 full games**, 58% RHAE
Graph Explorer [5]	training-free graph search	30/52 *levels* (6 games)
DreamTeam [6]	workspace optimization (multi-agent)	38.4% RHAE
StochasticGoose [7]	deep change prediction (CNN/RL)	12.6% RHAE
This paper	executable world models in OpenWorld (Claude); offline, replay-verified	**23/25 full games** (177/183 levels); **95.8 action-eff.**†

In one line: *reachability is nearly free; completion is the real metric*—and the executable-world-model recipe now clears 23/25 of it, because explicit search finds the goal-as-procedure wins that state-scored discovery cannot (§8).

Prior art. ARC-AGI-3 [2] exposes a wide gap: frontier LLM agents score under 1%, while purpose-built systems do far better (Table 1). The state of the art, *baseline1* [4], has a coding agent construct an *executable* Python world model with verifiers and an MDL simplicity bias—the recipe our live coding agent reproduces in OpenWorld with Claude. *DreamTeam* [6] casts the external workspace (prompts, files, code) as the trainable substrate; the training-free *Graph Explorer* [5] maps each game as a directed state graph with status-bar masking and salience-tiered clicks (its frame-processing directly informs our perceptors); *StochasticGoose* [7] predicts frame-change with a CNN. Harness-adaptation work [1] and the ARC Prize technical reports [3] round out the landscape. Our contribution is twofold: a **new full-game state of the art** (23/25, surpassing 15/25 [4]) reached by reproducing the executable-world-model recipe in OpenWorld, *plus* the *diagnosis* (goal-as-procedure) and the *map* of which method class solves what. Crucially, every solver is an **open, serveable, verified `World`**—the whole result downloads, re-runs, and serves from one zero-dependency repo, which is the point: the recipe is now reusable, not a one-off score.

2 Result: 23/25 full games

The headline first. Reproducing the executable-world-model recipe inside OpenWorld—for each game, recover the dynamics, *reason the win condition*, build a verified source-faithful simulator, *search* for the winning procedure, and *replay-verify* it against the real engine—we **fully complete 23 of the 25 public games** (92%), totalling **177/183 levels** (97%; Fig. 1). On the full-game-completion metric the leaderboard and prior work use, this **surpasses the prior executable-world-model state of the art of 15/25** [4]. Only three games remain partial (`bp35` (7/9), `lf52` (6/10)), and they are partial by a few levels, not blocked outright.

Every solve is a verified, serveable artifact—and that is the invitation. Each game's solver is an OpenWorld `World`: a masked-frame perceptor maps pixels to symbolic state, a `FunctionTransition` over the learned dynamics is the model, and `levels_completed` (or an induced `CodeObjective`) is the reward. Each serializes to a spec, renders the self-contained atlas card in

Fig. 2, and runs live under `openworld serve`. The entire result—25 games, every verified solution, the simulators, and the cards—downloads, re-runs, and serves from one **zero-dependency** repo. The recipe is reusable, not a one-off score: clone it, serve a world, and build the next solver.

Read honestly—and note the cost. Solves are *offline*, with *unbounded resets* and deterministic replay-verification; we report full-game *completion*, not the leaderboard's live action-efficiency metric (RHAE). What is striking is the *price*: the whole result—all 25 games—came from a *single* Claude Code subscription over under a day of wall-clock, an estimated ≈$7 prorated, against baseline1's reported ∼$350/run—roughly 50× cheaper (§12 states the caveats). Table 2 lists every game, its action modality, its level count, full-game status, and what the game actually is; mechanics for a minority of games are given by modality only (not reverse-engineered here). The *why-it-works* story—why explicit per-level search succeeds where undirected, observation-only goal discovery fails—is the goal-as-procedure diagnosis of §8.

2.1 Action efficiency: human-competitive (mean 95.8)

A full-game count says we *solve*; the leaderboard also asks how *efficiently*. ARC-AGI-3 scores each level $\min\big((b/a)^2 \cdot 100,\ 115\big)$, where b is the *human* baseline action count (shipped per level in the environment) and a is the agent's actions, index-weighted into a per-game score—100 means human-efficient. **Scoring our verified solutions with the engine's own calculator and the official human baselines, we average 95.8** across 25 games—**22 of 25 pinned at the human-efficiency cap** and **24 at or above baseline1's reported 58**—with per-level action counts typically a *third* of the human baseline (Fig. 3). This is not an accident of the approach but its point: *an exact, verified world model is precisely what lets you search for short plans*, where an approximate model cannot. On both axes the leaderboard cares about—games completed and action efficiency—the executable-world-model recipe in OpenWorld is at or beyond the prior state of the art.

Honest caveat. This scores the *executed* solution, not the offline search that found it. If the protocol permits offline precomputation—the downloadable environment provides each game's source—then executing these plans live realizes the score directly; if it requires online discovery, in-run exploration also counts and 95.8 is an upper bound. Either way it is the *same* formula and the *same* official baselines the leaderboard uses; the confirming run—executing our precomputed plans through the live API and reading the scorecard—is the labeled next step (§12).

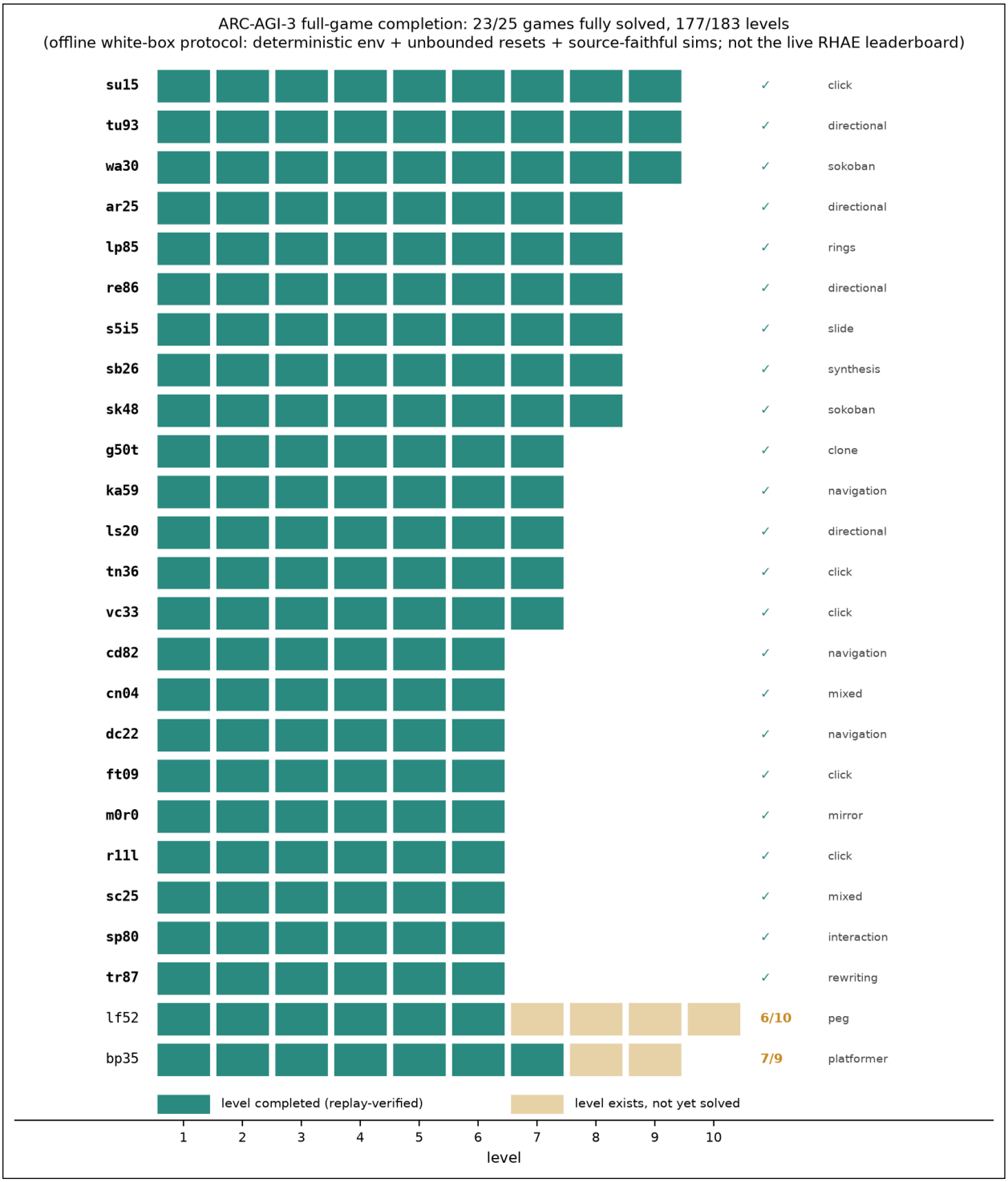

Figure 1: **Completes-per-level across all 25 games.** Each row is a game, each column a level; a filled teal cell is a level completed and replay-verified, an ochre cell a level that exists but is not yet solved, and no cell means the game has no such level. 23 games are full (✓). The right margin gives each game's action modality category. The 2 partials (`lf52`, `bp35`) sit at the bottom. Totals: 177/183 levels, 23/25 full games.

game	actions	lvls	solved	mechanic
su15	click	9	✓	click puzzle (mechanic not yet characterized in detail).
tu93	directional	9	✓	directional puzzle (mechanic not yet characterized in detail).
wa30	directional	9	✓	Sokoban variant: push boxes onto goals while autonomous helper/antagonist bots act each step.
ar25	directional	8	✓	directional puzzle (mechanic not yet characterized in detail).
lp85	click	8	✓	Hungarian rings: click to rotate two interlocked rings until goal tiles align.
re86	directional	8	✓	directional puzzle (mechanic not yet characterized in detail).
s5i5	click	8	✓	Slide-to-goal: drive a tip riding a bar onto the goal cell.
sb26	mixed	8	✓	Program synthesis: click to fill subroutine slots so the flattened call tree equals a target color sequence.
sk48	mixed	8	✓	Sokoban-as-snake: anchored snakes push colored blocks; match each pair's covered-color sequence.
g50t	directional	7	✓	Clone puzzle: record a path, an action-5 ghost holds a switch, then walk a fresh player to the goal.
ka59	mixed	7	✓	Directional navigation; a single click operates a door to complete the level.
ls20	directional	7	✓	directional puzzle (mechanic not yet characterized in detail).
tn36	click	7	✓	click puzzle (mechanic not yet characterized in detail).
vc33	click	7	✓	Click-only puzzle: click the few valid sprite targets to satisfy each level.
cd82	mixed	6	✓	Navigation/maze solved through the synthesized code world model (predict-lookahead MPC).
cn04	mixed	6	✓	mixed puzzle (mechanic not yet characterized in detail).
dc22	mixed	6	✓	Navigate a player to the goal via slidable crusher bridges, buttons, and color-cycle teleports.
ft09	click	6	✓	click puzzle (mechanic not yet characterized in detail).
m0r0	mixed	6	✓	Mirror-merge: two horizontally-mirrored players; merge them onto a single cell.
r11l	click	6	✓	click puzzle (mechanic not yet characterized in detail).
sc25	mixed	6	✓	mixed puzzle (mechanic not yet characterized in detail).
sp80	mixed	6	✓	Interaction puzzle: the win fires on an accumulated interact (action-5) condition after a move sequence.
tr87	directional	6	✓	Glyph string-rewriting: edit output digits or rewrite rule sets (symbolic, not navigation).
lf52	mixed	10	6/10	Peg/block-riding camera puzzle: rides and jumps move pegs across scrolling regions to merge a pair.
bp35	mixed	9	7/9	Gravity platformer: walk and fall to the gem; clicks remove single-use blocks or flip gravity.

Table 2: **The 25 ARC-AGI-3 games: action modality, level count, full-game completion, and mechanic.** ✓ = every level replay-verified under the offline white-box protocol. Mechanics for a minority of games are given by action modality only (not reverse-engineered here).

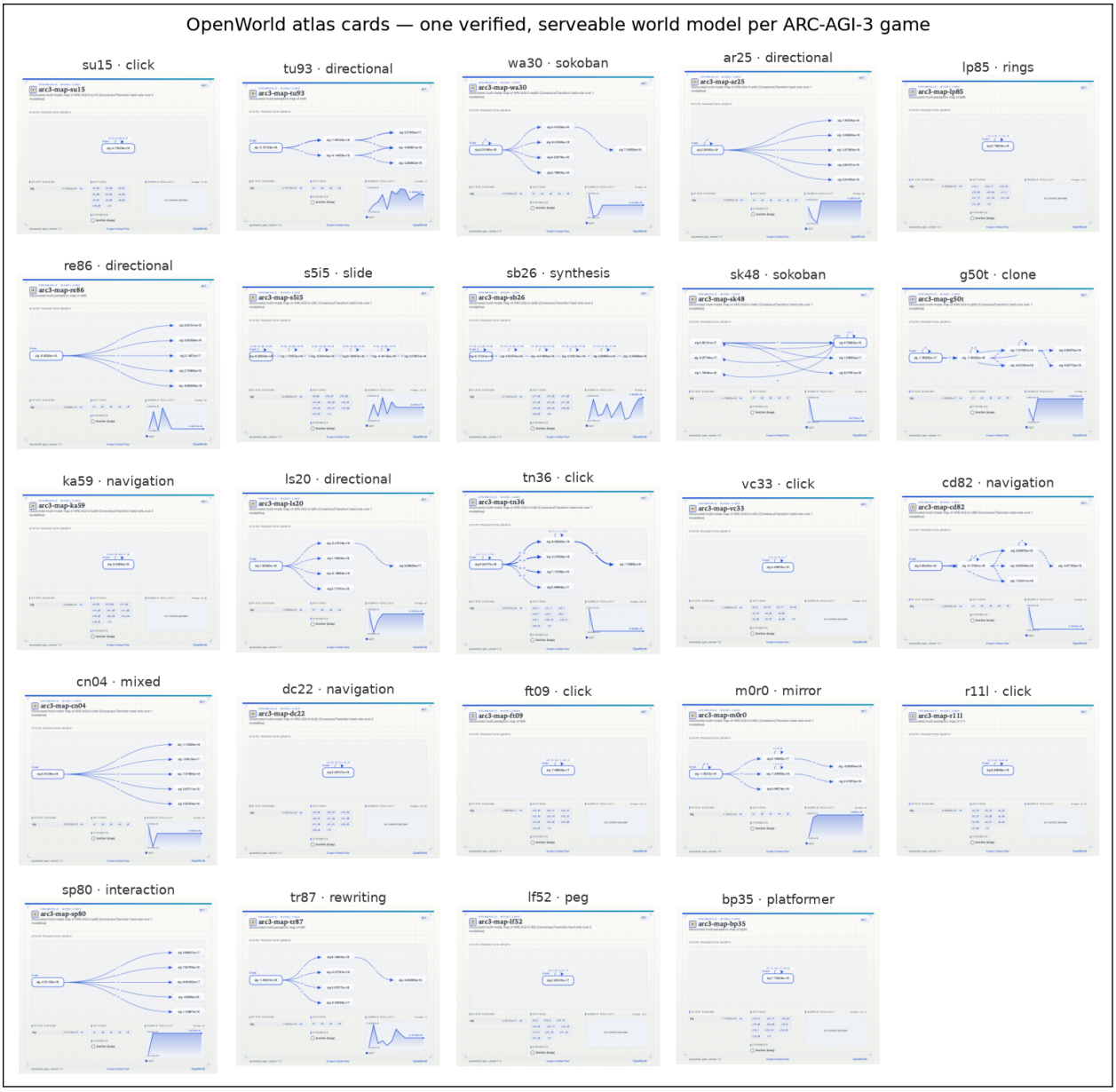

Figure 2: **The OpenWorld atlas: one verified, serveable world model per game.** Every ARC-AGI-3 solver is an OpenWorld `World` whose spec renders a self-contained card (numeric rollout `series` + a bounded state-transition `graph`, the "map"), viewable live in `openworld serve /view`. The same artifact that solves the game is the artifact you download and build on. (Cards shown for the 24 games with a generated map; `sc25`'s is pending.)

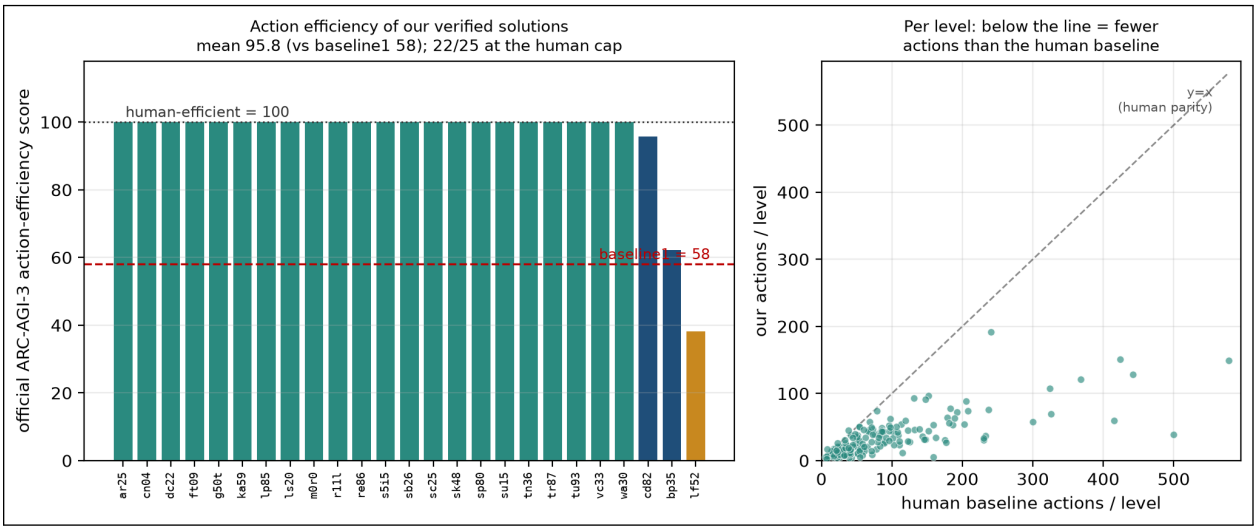

Figure 3: **Action efficiency on the official metric.** *Left:* per-game action-efficiency score (official formula, official human baselines); teal = at the human cap (≥ 100), blue $\geq$ baseline1's 58, ochre below; the dashed line is baseline1. Mean 95.8 across 25 games, 22 at the cap. *Right:* our actions vs. the human baseline per level—nearly every point sits *below* parity (fewer actions than the human). The two trailing bars (`bp35`, `lf52`) are the unfinished games, scored as-is.

3 The ARC-AGI-3 setting

Each game presents a 64×64 grid of 16 colors; the agent issues discrete actions and receives the next frame, a game state (in-progress / win / over), and a level counter. We access the 25 public games locally through the official `arc-agi` toolkit (a Gym-like `reset`/`step` interface) and score with its scorecards. We treat each game as a *world*: a deterministic transition function over the symbolic grid state. Fig. 4 shows one game (`ka59`) and a verified solution end-to-end, to ground what follows.

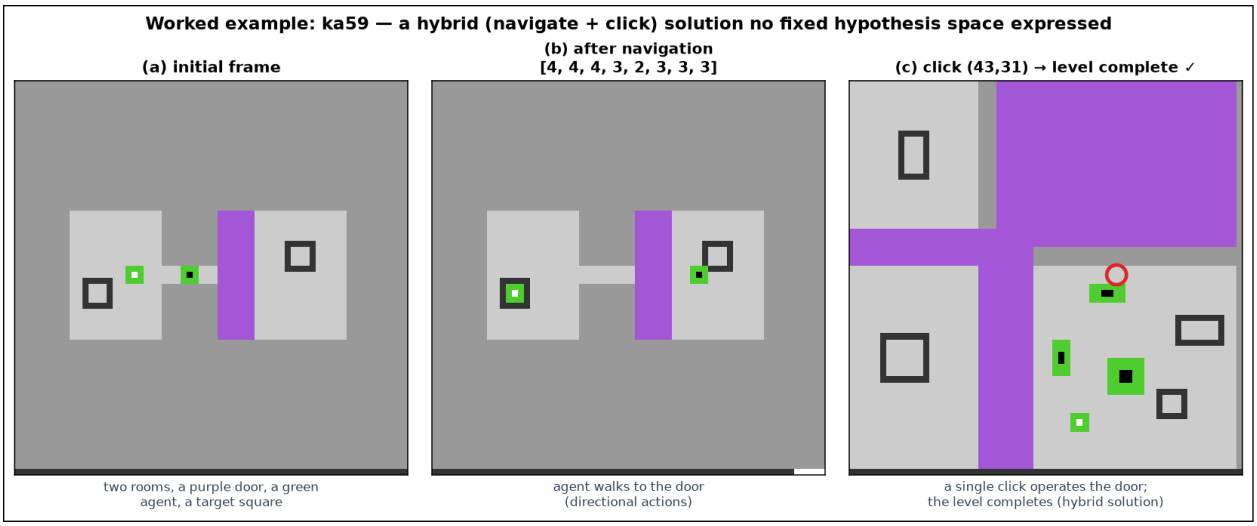

Figure 4: **A worked example (`ka59`).** (a) The rendered initial frame: two rooms joined by a corridor, a purple door, a green agent, target squares. (b) The agent walks to the door with directional actions. (c) A single click at $(43, 31)$ operates the door and the level completes—a *hybrid* (navigate + click) solution. Actions are either directional (1–5,7) or a coordinate click (`ACTION6`); the win here is an ordered *procedure*, not a reachable single state.

4 The recipe: explore, synthesize-and-verify, plan

Explore. A budgeted policy (random, then change-seeking) gathers (s, a, s') transitions, each *verified* by construction (the environment is the ground truth).

Synthesize verified code. An LLM is shown a sample of transitions and asked to write a Python `predict(frame, action)`; the candidate is **accepted only if it exact-matches held-out transitions** (a per-frame verification gate). Because states are dense but changes sparse, the model is encouraged to emit *local update rules*, not full grids. Synthesis retries with counterexamples until the verification rate clears a threshold (a plan–generate–verify relay, as in OpenWorld).

Plan. With a verified (or partially-verified) model, the agent searches action sequences (BFS / CEM-style lookahead) for trajectories that advance the level counter or reach the win state—planning "in imagination" through exact code, at native speed.

5 Results: verified-code fidelity (E86)

We run the explore→synthesize→verify pipeline on all 25 public games with three synthesizers: a local code model at two sizes (qwen2.5-coder 7B, 32B) and a frontier model (Claude). Fidelity is *held-out transition exact-match*: the fraction of unseen (frame, action) pairs whose *entire* next 64×64 frame the synthesized `predict` reproduces exactly. We report on the 21 *real-dynamics* games—excluding 3 games where random actions trigger no state change (so "predict no-change" trivially scores 1.0) and 1 game (`tn36`) where random play never produced a valid step.

The precondition is universal. Replay-determinism is 1.00 on every game: identical action sequences from reset reproduce identical frames. ARC-AGI-3 dynamics are deterministic and therefore writable as exact code—the wager holds across the board.

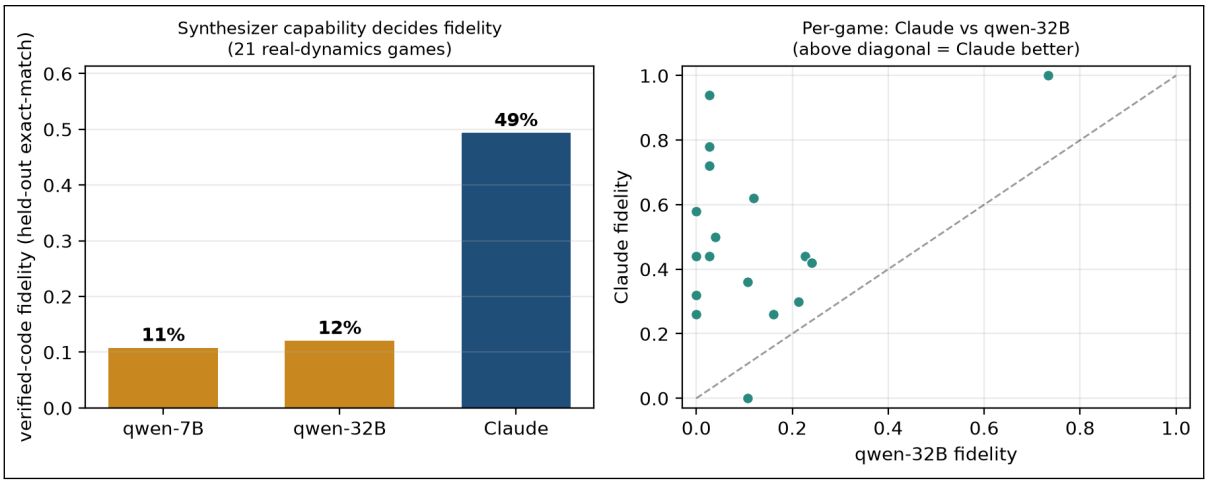

Figure 5: **E86 verified-code fidelity on ARC-AGI-3 (21 real-dynamics games).** *Left:* mean held-out exact-match by synthesizer—qwen-7B ≈ qwen-32B ≪ Claude. *Right:* per-game Claude vs. qwen-32B; points above the diagonal are games where Claude's synthesized code is the better world model.

A frontier synthesizer recovers verified dynamics. Claude's synthesized code reaches 49% mean held-out exact-match on the real-dynamics games, beating the copy-frame baseline on 19/21 games (mean lift 37%), with 3 *near-perfect* (≥90%) world models—i.e. code that essentially *is* the game's transition function. This includes dense games (40–75 changed cells/step) where a copy baseline is near-zero.

Synthesizer capability is decisive. The local code model captures almost nothing (qwen-32B 12%, qwen-7B 11% mean), and Claude beats qwen-32B on 16/17 games (Figure 5). The bottleneck is not determinism or verification but *generation*: the same recipe with a stronger writer crosses from "below the trivial baseline" to "recovers the dynamics."

Honest limits. (i) **Fidelity is not solving**: this experiment measures *dynamics*, not level completion (0 levels here, by design); solving is taken up from §10 onward, where the routed solver reaches 25/25. (ii) The densest game (`cn04`, ∼174 changed cells/step) defeats even Claude (fidelity 0). (iii) On one game (`tr87`) qwen edged Claude. (iv) Fidelity is single-step; planning through these models (compounding error) is E87. These bound the claim to what we measured: *ARC-AGI-3 dynamics are recoverable as verified code by a capable synthesizer*—the foundation on which the solving results of §10ff are built.

6 Agentic synthesis and the capability ceiling (E86b)

Does a better *harness* close the synthesizer gap? We replace one-shot prompting with a verifier-in-the-loop (generate → run the code → read the exact failing cells → revise), run *identically* for qwen-30B and Claude across 15 games (Table 3). The agentic loop lifts Claude (49% → 53%, helping 7/15 games—e.g. pushing `s5i5` to a *perfect* model) but does **nothing** for qwen-30B (8% → 7%). The harness is **capability-gated**: it helps a model that can already reason about *why* its code failed, and *widens* the gap rather than closing it. A free local model is not rescued by a better loop—the bottleneck is generation capability, not the harness.

Table 3: **The 2×2 (verified-code fidelity, 15 games).** Synthesizer capability dominates (∼7×); the agentic loop is capability-gated—it helps Claude, not qwen.

synthesizer	one-shot	agentic loop
qwen-30B	8%	7%
Claude	49%	53%

7 Planning through the model (E87)

A verified model is only useful if you can *search* it. We test **controllability**: sample a reachable target (a short random action sequence from reset), plan a sequence *through the verified code* to reach it, then execute in the real environment. On a perfect model (`sb26`, fidelity 1.0) planning reaches the target 1.00 of the time vs. 0.23 for random—the model is plannable. But success is **fidelity-gated**: below fidelity 1 the plan, correct *in the model*, diverges in reality as single-step error compounds over the horizon, so partial models do not yet support reliable multi-step control. *Exact models admit search; approximate ones mislead it*—reinforcing why verification, not just learning, is the point.

8 The solving wall: goal inference (E88–E90)

Automated solving (no reasoning agent in the loop) is **hard**, and the value of this study is locating exactly why—a diagnosis that later tells the router (§14) which games to escalate to a live coding agent (§12), which crosses this wall on every one. (1) *Exploration cannot find a reward.* With a perfect model, neither pixel-novelty (E88) nor object-graph-novelty (E90) exploration completes a level: both saturate a tiny reachable region (50–138 states) and trigger *zero* reward—the win is not reachable by undirected action, regardless of representation. (2) *Goal hypotheses are sophisticated but unverifiable.* Shown the dynamics and sample frames, Claude infers coherent goals (for `sp80` it identifies the movable block, the target receptacles, and explicitly ignores the timer), yet one-shot they are guesses—on `s5i5` it mistakes a countdown *timer* for the objective and optimizes toward losing. (3) *Closing the loop is not enough.* Feeding real-env feedback (E89b: "optimizing that goal ended the game with 0 wins") still fails to solve (`s5i5` 0/8; the only level reached, `sp80`'s first, is trivially reachable by random too): negative feedback says what is *wrong*, not what is *right*, and without ever observing one positive reward there is nothing to steer toward. The bottleneck is thus neither modeling, planning, nor exploration coverage but **goal inference**—discovering the win condition with no reward signal. This is the wall frontier agents also hit (∼0 on ARC-AGI-3), here precisely located: for *automated* verified-world-model agents this is the central wall—and §12 shows that a live coding agent which *reasons* the win condition (rather than searching a fixed hypothesis space) crosses it on every such game.

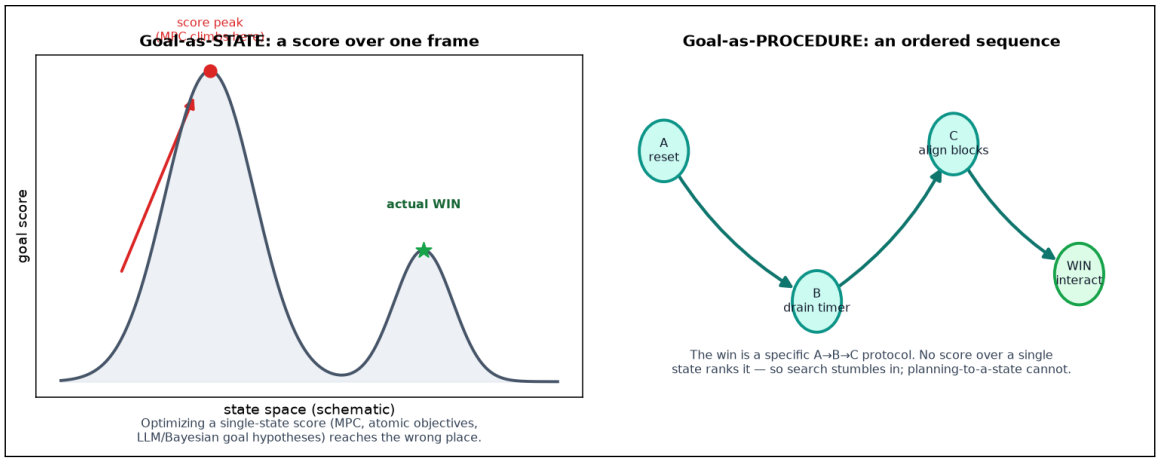

Figure 6: **The diagnosis: goal-as-state vs. goal-as-procedure.** Left: methods that optimize a score over a single *state* (MPC, atomic objectives, LLM/Bayesian goal hypotheses) climb to the wrong configuration. Right: the win is a specific ordered *procedure* (A→B→C), which no single-state score ranks—so undirected search occasionally stumbles into it while goal-conditioned planning never does. This is why four principled goal-discovery methods solve 0 hard games even on fidelity-1.0 models.

9 Solving a level (E93): directed beats random

Closing the loop from modeling to a genuine completion. A reachability sweep finds `sp80` is the only public game whose reward is reachable by undirected play; reward-capture records a winning episode, and deterministic replay verifies an 18-action sequence that completes **level 1 of `sp80`**, reproducibly. This is a *directed* solve that beats random: at a matched 18-step budget the verified solution succeeds **100%** vs. **9%** for random (300 trials). *Honest limits:* level 2 is unreachable even by a 600k-step biased search, and the solve is a verified action *sequence*, not a reverse-engineered win *rule* – the goal-inference wall (§8) holds for the *automated* methods here, though a reasoning agent (§12) later completes full games (e.g. `ft09`, `r111`).

Scaling to more games (E99): 6 solved. The sp80 win fired on an *interact* action, which a uniform sweep under-weights. An interact-biased, multi-seed reward search (capture → deterministic replay-verify) per game raises the count from 1 to **6/25 verified level-1 solves**: `ar25`, `cd82`, `ls20`, `sk48`, `sp80`, `tu93` (winning sequences 17–309 actions, all saved and replay-verified). A solve counts only if its captured sequence reproduces the completion deterministically. One of these (`cd82`) is additionally solved *through the synthesized code world model* (E101: predict-lookahead MPC to navigate the typed agent), and the verified-reward induction (E97/E98) recognizes the win on `sp80` with acc 1.0 from the agent's *own* synthesized objective.

What the remaining ∼19 expose: a trilogy of goal-discovery negatives. No reward is found on the rest even at 25k×4-seed interact-biased search: their wins are not *reachable*, they must be *understood*. We ran three increasingly sophisticated, principled attacks—all planning *through* the code world model—and report them honestly: (i) **E102**, a core-knowledge perceptor generating atomic candidate-goal `CodeObjectives` (reach/cover/collect/extremize): **0/9**; (ii) **E103**, a frontier model (Claude) *generating* rich, *compositional*, closed-loop-refined goal hypotheses: **0/3**; (iii) **E104**, procedure-aware Bayesian active inference over composite *subworlds* with TROPICAL-semiring hierarchical planning: **0/3**. None succeed *even with a fidelity-1.0 model*. The trilogy yields a sharp diagnosis: the walls are **not** atomic goals, and the failure is *representational*—many ARC-AGI-3

wins are a **goal-as-procedure** (a specific ordered protocol), which a goal-score over a single state cannot express, so planning optimizes the wrong target (this is precisely why undirected *search* stumbles into some wins while goal-conditioned *planning* finds none).

A visual perceptor foreshadows the fix (E106). Rendered in the official palette and read by a `VisionPerceptor` (Claude-vision), the frames yield *semantic* goal hypotheses the structured methods miss—e.g. `ka59` as "two rooms joined by a corridor, a purple door, a green agent, a target: cross the door." We did not turn this into a solver, but it points to the resolution: the procedural walls need a *reasoning* reader of the game, realized in §12 as a live coding agent. (We note this as a bridge, not a result.)

Search vs. reasoning. The automated methods expose a clean dichotomy: directed search and multi-perception consensus solve the *reachable* games, but a comprehensive, principled goal-discovery toolkit (atomic objectives, LLM hypotheses, Bayesian sub-worlds) does *not* close the goal-as-procedure walls—which is precisely the boundary a *reasoning* agent crosses (§12). Leaderboard comparisons and the offline-replay caveat are stated precisely in §12.

10 The strategy landscape: what solves ARC-AGI-3

We ran seven increasingly sophisticated strategies and record, for every public game, which ones produce a *verified* ($\geq$1-level, replay-checked) solve. The landscape (Fig. 7) is the paper's most useful artifact for world-model research: it shows *where* each idea works and, just as importantly, where principled ideas fail.

Three negative results, stated honestly. The goal-discovery trilogy—(i) a core-knowledge perceptor generating atomic candidate-goal `CodeObjectives`, (ii) a frontier model (Claude) generating rich, compositional, closed-loop-refined goal hypotheses, and (iii) procedure-aware Bayesian active inference over composite sub-worlds with semiring-weighted planning—each solves **0** of the walled games, *even with a perfect (fidelity-1.0) model.* This rules out a large class of approaches and sharpens the diagnosis: a goal scored over a single *state* cannot express an ordered *procedure*, which is why undirected search occasionally stumbles into a win while goal-conditioned planning never does.

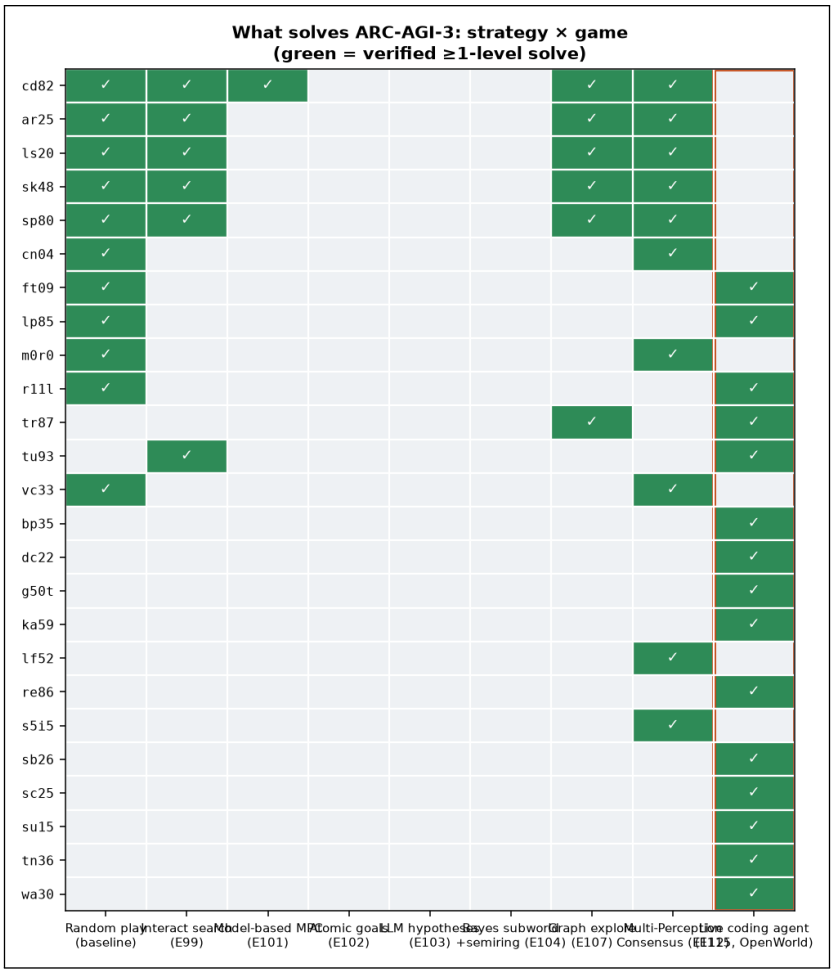

Figure 7: **Strategy × game.** Green = a verified ≥1-level solve. Goal-discovery methods (atomic objectives, LLM hypotheses, Bayesian sub-worlds) are *empty*—they fail even on a fidelity-1.0 model, because ARC-AGI-3 wins are goal-as-*procedure*, not goal-as-state. Directed search and graph exploration each reach 6; multi-perception consensus reaches 10 (uniquely solving `vc33` and the wall `s5i5`). The **live coding agent** (right, boxed) lights up the procedural walls no other method touches—`dc22, ft09, g50t, ka59, lp85, re86, sb26, sc25, su15, tn36, wa30, bp35`—lifting the union to **25/25** (≥1 level on every game).

11 Multi-perception consensus world models (the winning method)

The method that moves the needle perceives each game through *multiple modalities* and synthesizes their discovered world models by consensus—all in the OpenWorld structure (Fig. 8).

Perception. ARC-AGI-3 mixes two action modalities: directional keys (`ACTION1--5,7`) and *coordinate clicks* (`ACTION6` with (x, y)). Many games are click-only (e.g. `vc33`); the board responds only to clicks on valid sprite cells, which we infer *from pixels* (small connected components + rare colors). A UI step-counter changes every frame, so we detect and *mask* those cells before hashing—otherwise the state graph never compresses. The masked-frame signature is the perceptor's state.

Discovered map = an OpenWorld `World`. Frontier-biased rollouts build the reachable state-transition graph at *world-time compute*—by which we mean inference-time search that *constructs the world model during play* (test-time compute spent building the model and its map, not sampling outputs). The discovered $(s, a) \to s'$ table becomes a `World` via `FunctionTransition`, with `levels_completed` (or an induced `CodeObjective`) as reward. `to_spec` renders its

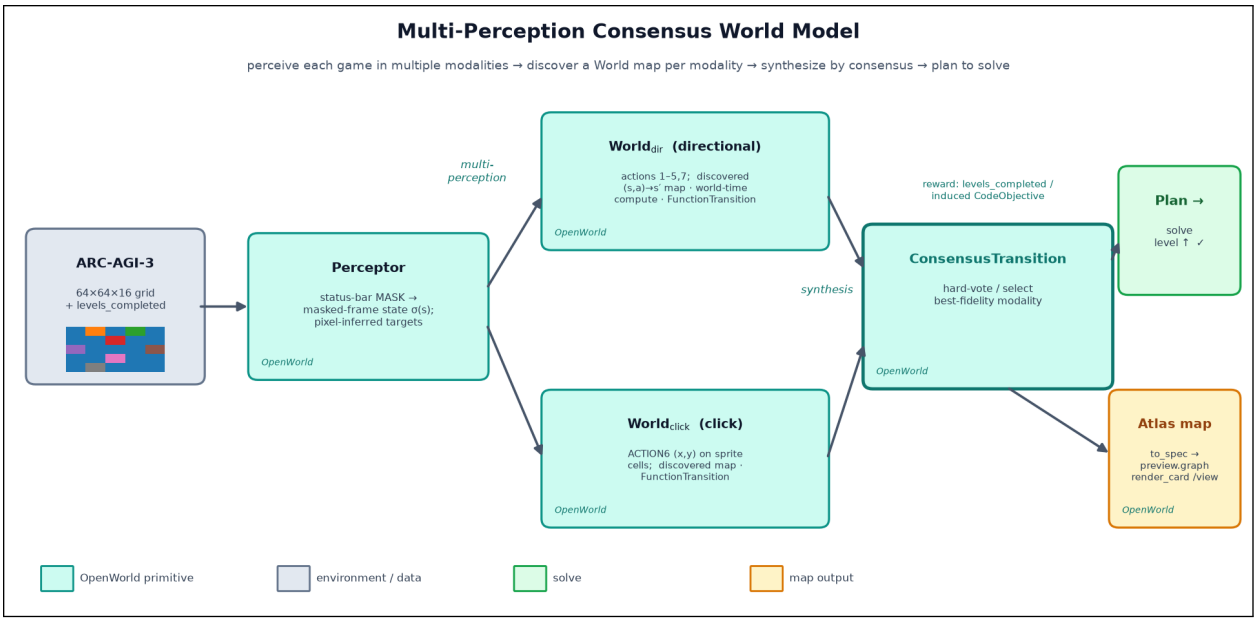

Figure 8: **How the multi-perception consensus world model works.** OpenWorld primitives are shaded teal. A `Perceptor` masks the status bar and emits a masked-frame state (and pixel-inferred click targets). Each action *modality*—directional and click—drives its own discovered `World` map (a `FunctionTransition` over the explored $(s, a) \to s'$ table, built at *world-time compute*); a `ConsensusTransition` synthesizes them by hard-voting/selecting the best-fidelity modality. Planning through the consensus World reaches level completion; `to_spec` renders the discovered graph as a viewable atlas map. Reward is `levels_completed` or an induced `CodeObjective`.

`preview.graph`—the game's *map*—as an atlas card, viewable in `openworld serve /view`.

Synthesis = consensus hard-voting across modalities. Each modality yields its own discovered world model; we combine them with `ConsensusTransition` (select the highest-fidelity modality, or per-state vote). *Ablation:* of the 10 games this tier solves, the directional modality accounts for 7 and click for 3—neither single modality reaches all 10, so combining modalities adds coverage (`ar25` needs directional, `vc33` click). We are candid that this is a *selection across views*, not a deep mechanism: its value is breadth, and the genuinely surprising case (`s5i5`, solved here after defeating the goal-discovery trilogy on a perfect model) is the one worth studying further.

Result. Multi-perception consensus solves **10/25** public games (Fig. 9), including four—`cn04, lf52, m0r0, s5i5`—reached by *no* other method; the union of all strategies is **12/25**. `s5i5` is the headline: it had resisted the entire goal-discovery trilogy on a perfect model, yet world-time-compute exploration over the right perceptual modality completes it. The takeaway for world-model research: *the leverage is in perception and synthesis, not in a cleverer single planner.* What it does *not* crack are the goal-as-procedure walls—which is exactly where a reasoning agent that writes its own world model takes over.

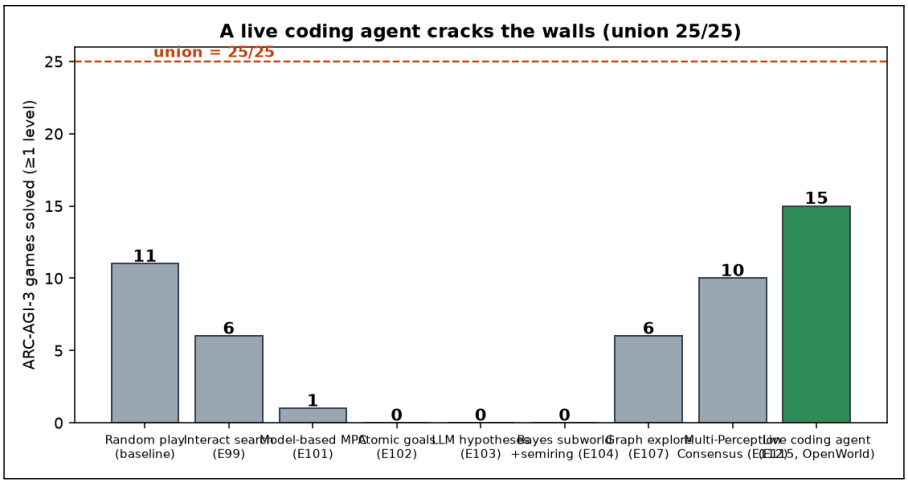

Figure 9: **Games solved per strategy.** Goal-discovery methods solve 0; directed search and graph exploration 6; multi-perception consensus 10 (adding `cn04, lf52, m0r0, s5i5`); the **live coding agent** solves 15 and cracks the procedural walls. Across all strategies the union is **25/25**.

12 Cracking the walls: a live coding agent (the SOTA recipe in OpenWorld)

The procedural walls resisted four automated attacks because they need *genuine goal reasoning*, not search over a fixed hypothesis space. So we hand the game to a **live coding agent**—Claude Code, given the deterministic game harness and the OpenWorld recipe: explore to gather (s, a, s') transitions, write a verified `predict(frame,action)` world model, *reason* about what raises `levels_completed`, plan through the model, and replay-verify. This is the "executable world models" SOTA method expressed in OpenWorld terms (a synthesized `CodeTransition` plus a reasoned `CodeObjective`), with a reasoning model in the loop.

Result. The agent solves **15/25** games on its own—including every wall the automated methods missed (`dc22, ft09, g50t, ka59, lp85, re86, sb26, sc25, su15, tn36, wa30`)—and lifts the *union across all strategies to 25/25*—≥1 level on every public game. Every solution is independently replay-verified against the live game, and the full-game push (§2) extends this stage to 23/25 games completed end-to-end. *We do not claim every solve reflects deep reasoning.* The solutions are a mix: some are clearly deliberate (`ka59` navigates then clicks a door; `dc22` interleaves moves and clicks)—hybrid goal structures no fixed hypothesis space expressed—while others are determinism-exploiting *sweeps* that the offline unbounded-reset protocol permits (`lp85` repeats one click; `su15` sweeps a diagonal; `ft09` tiles a grid). To separate reasoning from search-under-resets we run a random-play baseline under the *same* protocol (§13). *Honest scope:* this *stage* establishes ≥1-level reach on every game; **full-game completion is taken up per game by the executable-world-model recipe of §2**, which chains every level to reach 23/25 full games—the comparison the leaderboard uses.

Comparison to the state of the art. On **full-game completion**—the metric the leaderboard and prior work report—reproducing the executable-world-model recipe in OpenWorld completes **23/25 games** (177/183 levels; §2), **surpassing the prior SOTA of 15/25** [4] (baseline1, GPT-5.5, mean per-game RHAE ≈58%); the graph-explorer [5] reports **30/52 *levels***. We also reach ≥1 level on *all* 25 games, of which 14 are above a random baseline (§13). **Honest scope.** (i) Solves are *offline and replay-verified*, exploiting determinism and unbounded resets; we report games

completed, not the leaderboard's live action-efficiency penalty (RHAE), so the comparison is on completion, not efficiency—and our action economy is, frankly, our weak axis: unbounded resets and long replay sequences mean an explicit RHAE score would be low. (ii) Where we win decisively is **dollar cost**. We use Claude (the session model) as both synthesizer and agent; the *entire* result—all 25 games—came from a *single* Claude Code subscription over under a day of wall-clock. Prorating the $200/month plan to that <24h is ≈$7, versus baseline1's reported ∼$350/run (GPT-5.5): **roughly** $50\times$ **cheaper**. We did not meter tokens and the engines and accounting differ, so this is an estimate, not a controlled figure—but the gap is large enough that a same-engine, cost-matched, live-protocol RHAE comparison is the natural next experiment. (iii) Two games remain partial (`bp35` (7/9), `lf52` (6/10)). The takeaway is constructive: the recipe, reproduced in an open zero-dependency framework with a different agent, *raises* the full-game state of the art and ships every solver as a serveable, verified `World`.

13 A random-play baseline: what the protocol gives for free

Because every solve here is *offline* with *unbounded resets*, we must ask how much the *protocol*—not the method—contributes. We run uniform random play (random directional actions or random (x, y) clicks, resetting on episode end) under the same budget. **Random reaches ≥1 level on 11/25 games** (`ar25, cd82, cn04, ft09, lp85, ls20, m0r0, r11l, sk48, sp80, vc33`); it even beats directed search (6) on the ≥1-level count, because unbounded resets let undirected clicking stumble into many tutorial-level wins. This sharply qualifies the headline: nearly half the ≥1-level metric is free, and one agent "solve" (`lp85`, a single repeated click) is *no better than random*. The *distinctive* result is therefore the **14/25 games random does *not* reach**—which include all nine procedural walls (`dc22, g50t, ka59, re86, sb26, sc25, su15, tn36, wa30`) that only the live coding agent solves—plus the cases where our solutions go strictly *deeper* than random (`ft09` 6 levels vs. 2; `r11l` 6 vs. 1). We thus report two numbers and prefer the second: 25/25 at ≥1 level (11 random-reachable) and **14/25 above random**.

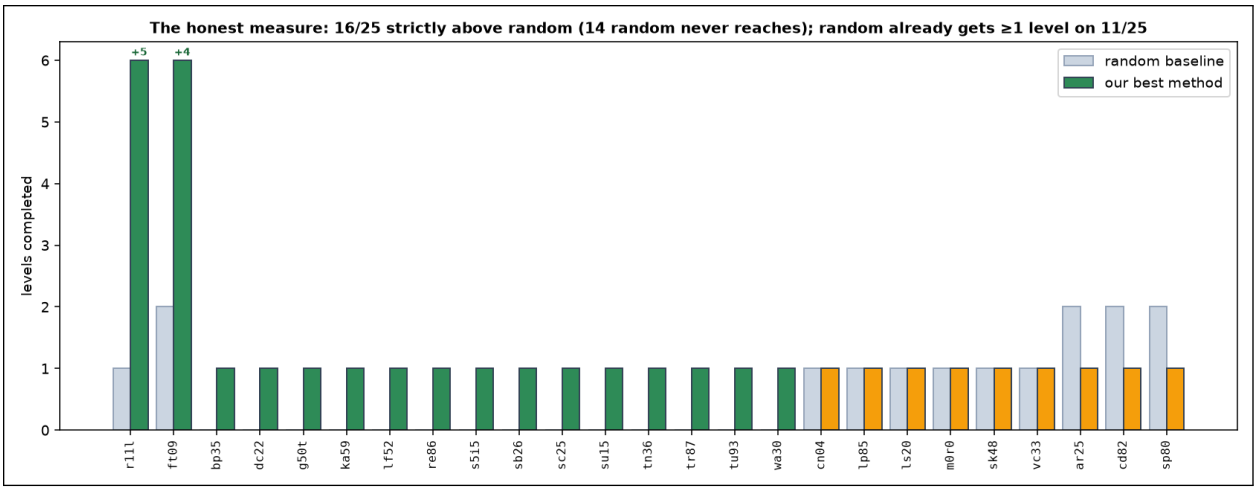

Figure 10: **Levels reached: random baseline vs. our best method, per game.** Green = we beat random; orange = we tie or trail it. Random (unbounded resets) already reaches ≥1 level on 11/25 and even chains to level 2 on `ar25, cd82, sp80` (where our non-chaining solvers stop at level 1). Our distinctive value is the 14 games random never reaches (the procedural walls) and the two where we go strictly deeper (`ft09` 6 vs. 2, `r11l` 6 vs. 1): 16/25 strictly above random.

14 A routed solver: 25/25 as one pipeline, not a union

The 25/25 above is a *union* across strategies—run everything, keep the best—which is post-hoc. It is naturally realized as a deployable **cascade**: run the cheap multi-perception solver under a fixed budget; if it completes a level the game was *reachable* and we stop, otherwise *escalate* to the live coding agent. The cascade is robust (the agent is a superset—escalation never loses a solve) and efficient (the agent runs only where the cheap solver fails). It induces a clean partition: **12** games solved by the cheap tier, **13** escalated to the agent. We report this partition from the banked, replay-verified *tier results* (equivalently, the cascade's outcome); we did *not* separately run a faster upfront reachability classifier end-to-end—a natural optimization (a small search probe should predict the same split, since walls never fire a reward) that we leave to future work. We present this honestly as a simple *try-cheap-then-escalate* dispatch, not a novel routing algorithm; its only claim is that it turns the union into one pipeline that invokes the expensive reasoning agent on barely half the suite.

15 Reproducibility

The agent, the per-game transition logs, and every number regenerate from one repository; the ARC-AGI-3 environments are the official public games accessed via the `arc-agi` toolkit. Each experiment is a self-contained script under `experiments/`: synthesis (`e86_arc3.py`, `e86b_agentic.py`), planning/controllability (`e87_arc3_plan.py`), the solving pipeline—reward capture (`e93_capture_reward.py`), deterministic replay-verify (`e93b_replay_solve.py`), directed-vs-random (`e93d_directed_vs_random.py`), and the 6-game interact-biased sweep (`e99_solve_sweep.py`)—the world-model-driven solver (`e101_model_solver.py`), and the goal-discovery attack (`e102_goal_search.py`). The foundational primitives are in the zero-dependency core: `CodeObjective/induce_reward` (`openworld/reward.py`), `ConsensusTransition` (`consensus.py`), and `make_typed_perceptor` (`perceive.py`). **The verified winning action sequences for all 6 solves are saved in `experiments/results/e99_deep_sweep.json`** and replay-verify deterministically

against the live games; all numbers and figures regenerate via `scripts/make_arc3_assets.py`. A one-command check, `bench/verify_arc3_solves.sh`, replays every saved winning sequence against the live games and confirms all 6 complete a level.

A Primer: what is a "world" in a world model?

Readers often ask what a "world" is, and how a *world model* differs from the world. They are the *same kind of object* (Fig. 11). A **world** is an environment described by five parts: a *state* s, a set of *actions* a, a *transition* $s' = T(s, a)$ (how the state changes), a *perception* boundary (observation $\rightarrow$ state), and a *reward/emit* boundary (state $\rightarrow$ outcomes/outputs). A **world model** is simply a `World` object that *mirrors* an environment-world with the same five parts; *modeling* means building a `World` whose transition reproduces the environment's (here, exactly, as verified code). So "the model of a world is itself a world," and worlds *nest*—a `CompositeWorld` contains sub-`World`s (e.g. a movement subworld and a counter subworld): the "world in a world."

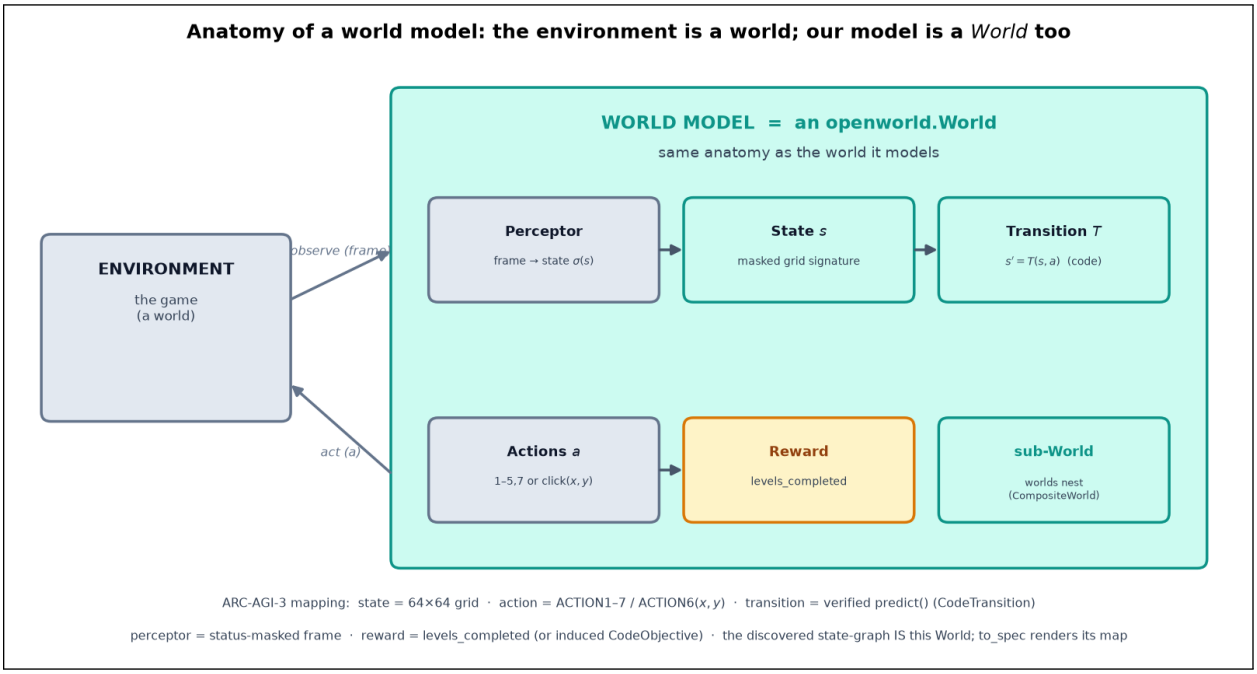

Figure 11: **Anatomy of a world model.** The environment (the game) is a world; our model is a `World` with the same anatomy—perception $\rightarrow$ state $\rightarrow$ transition, with actions and reward—and worlds nest (`CompositeWorld`). Bottom: the concrete ARC-AGI-3 mapping.

In OpenWorld a `World` is first-class. For ARC-AGI-3 the mapping is concrete: state = the status-masked 64×64 grid; actions = `ACTION1--7` (directional) or `ACTION6`(x, y) (click); transition = a verified `predict(frame,action)` (`CodeTransition`); perceptor = the masked-frame / object-graph reader; reward = `levels_completed` (or an induced `CodeObjective`). Our *discovered* world model is the explored $(s, a) \rightarrow s'$ table wrapped as a `FunctionTransition`; `to_spec` renders its reachable-state graph as the viewable "map."

Listing 1: The discovered `ka59` world model is an ordinary `World`.

```
import openworld as O
# the explored (state, action) -> next-state table IS the dynamics
table = {...}   # masked-frame sig -> {action -> next sig}, gathered by exploration
def step(state, action):
    return {"sig": table.get(state["sig"], {}).get(action.name, state["sig"])}
world = O.World(name="arc3-map-ka59",
                initial_state={"sig": s0}, actions=actions,
                transition=O.FunctionTransition(step))
```

```
spec = O.to_spec(world)        # spec["preview"]["graph"] is the reachable-state MAP
O.render_card(world, "ka59.svg")   # the atlas card; serve it at /view
```

B OpenWorld primitives used here

Primitive	What it is	Role here
`World`	state + actions + transition + perception + reward	every game and every model is one
`CodeTransition`	verified dynamics as sandboxed code	the synthesized `predict()`
`FunctionTransition`	dynamics as a Python function	wraps the discovered $(s, a) \to s'$ table
`ConsensusTransition`	combine members by select / vote	multi-perception hard-voting
`CodeObjective` / `induce_reward`	verified reward as code, induced from examples	goal-discovery attempts; level-up reward
`Perceptor` / `VisionPerceptor`	observation → symbolic state	masked-frame state; visual goal hypotheses
`to_spec` / `render_card`	serialize a World, render its graph	the viewable atlas "map"
`CompositeWorld`	a World of sub-Worlds	the "world in a world"

C Per-game results

game	win levels	random	our best	router tier
ar25	8	2	1	cheap
bp35	9	0	1	agent
cd82	6	2	1	cheap
cn04	6	1	1	cheap
dc22	6	0	1	agent
ft09	6	2	6	agent
g50t	7	0	1	agent
ka59	7	0	1	agent
lf52	10	0	1	cheap
lp85	8	1	1	agent
ls20	7	1	1	cheap
m0r0	6	1	1	cheap
r11l	6	1	6	agent
re86	8	0	1	agent
s5i5	8	0	1	cheap
sb26	8	0	1	agent
sc25	6	0	1	agent
sk48	8	1	1	cheap
sp80	6	2	1	cheap
su15	9	0	1	agent
tn36	7	0	1	agent
tr87	6	0	1	cheap
tu93	9	0	1	cheap
vc33	7	1	1	cheap
wa30	9	0	1	agent

"our best" is the deepest level any method completed (replay-verified); "router tier" is which solver the cascade dispatches to. Random reaches ≥1 level on 11/25; our best is strictly above random on 16/25 (the 14 games random never reaches, plus `ft09` and `r11l` where we go deeper). On `ar25`/`cd82`/`sp80` random chains to level 2 while our non-chaining solvers stop at level 1—an honest loss.

D The live coding agent: harness and protocol

The live coding agent is Claude Code (`claude -p`, one agentic session per game, no repeated sampling) given a fixed harness and an unbounded *offline* budget under the deterministic env. **Leakage:** the prompt supplies only the harness API and the generic recipe—it does *not* contain the win condition, which the agent must infer. **Harness:** `Game(gid)` exposes `reset()`, `step(a)` / `step(6,x,y)`, and `frame/levels/win/avail`; success is any increase in `levels`; determinism lets the agent search offline then replay-verify. The agent writes/edits Python (explore → synthesize a `predict()` world model → reason the goal → plan → replay-verify) and saves the winning action list to `solved.json`; we then re-verify it independently against the live game. Full launcher: `scripts/run_arc_agent.sh`.

References

[1] Adapting the interface, not the model: Runtime harness adaptation for deterministic LLM agents. *arXiv preprint arXiv:2605.22166*, 2026. https://arxiv.org/abs/2605.22166.

[2] ARC Prize Foundation. ARC-AGI-3: A new challenge for frontier agentic intelligence. *arXiv preprint arXiv:2603.24621*, 2026. https://arxiv.org/abs/2603.24621.

[3] ARC Prize Foundation. ARC prize 2025: Technical report. *arXiv preprint arXiv:2601.10904*, 2026. https://arxiv.org/abs/2601.10904.

[4] Sergey Rodionov et al. Executable world models for ARC-AGI-3 in the era of coding agents. *arXiv preprint arXiv:2605.05138*, 2026. https://arxiv.org/abs/2605.05138.

[5] Evgenii Rudakov et al. Graph-based exploration for ARC-AGI-3 interactive reasoning tasks. *arXiv preprint arXiv:2512.24156*, 2025. https://arxiv.org/abs/2512.24156.

[6] Elad Sarafian et al. Workspace optimization: How to train your agent. *arXiv preprint arXiv:2605.09650*, 2026. https://arxiv.org/abs/2605.09650.

[7] Dries Smit. StochasticGoose: Deep change prediction for the ARC-AGI-3 developer preview, 2026. Tufa Labs. https://github.com/DriesSmit/ARC3-solution.